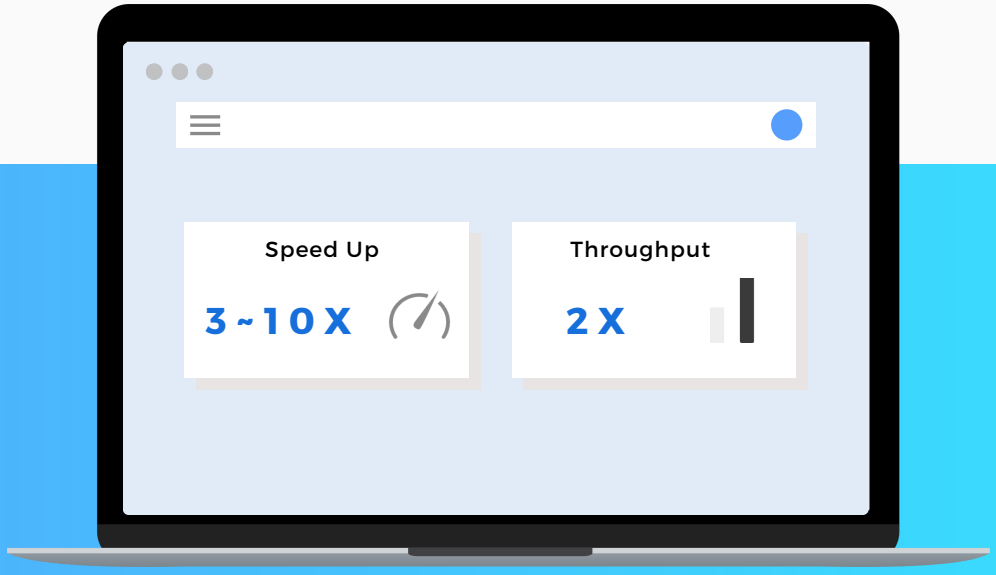
MODERN DATA PLATFORM EXCELLENCE SERIES

ALLUXIO

The Presto Optimization Handbook



Best Practices & Tuning Tips

INTRODUCTION

In today's data-driven world, your ability to extract rapid insights and make informed decisions depends on a high-performance data querying platform. PrestoDB, or simply Presto, is a popular open-source distributed SQL query engine that has become the go-to tool for organizations running analytics across various data sources.

However, many Presto users and developers still face challenges with inconsistent query performance. Unlocking Presto's full potential is crucial for your organization to derive maximum value from its data. Faster queries result in enhanced user experiences, quicker insights, increased resource efficiency, and reduced costs.

This comprehensive guide aims to equip data platform engineers, data engineers, Presto administrators, developers, and advanced users with the knowledge and best practices necessary to maximize Presto query performance. You will learn a holistic approach for Presto optimization, covering tuning process, query planning, caching, data formats (Parquet, ORC), table formats (column layouts), data sources (S3 and more), and data platform architecture related to Presto performance.

By the end of this ebook, you will have a deeper understanding of:

- How Presto runs queries under the hood
- What happens during query execution and the bottlenecks that can impact query performance
- How to refine Presto for optimal query performance
- The tuning steps and seven best practices for maximizing Presto query efficiency, including configuration settings, session properties, SQL statement and practical advice
- Presto optimizations at Uber on large-scale HDFS
- Presto on hybrid cloud at WalmartLabs

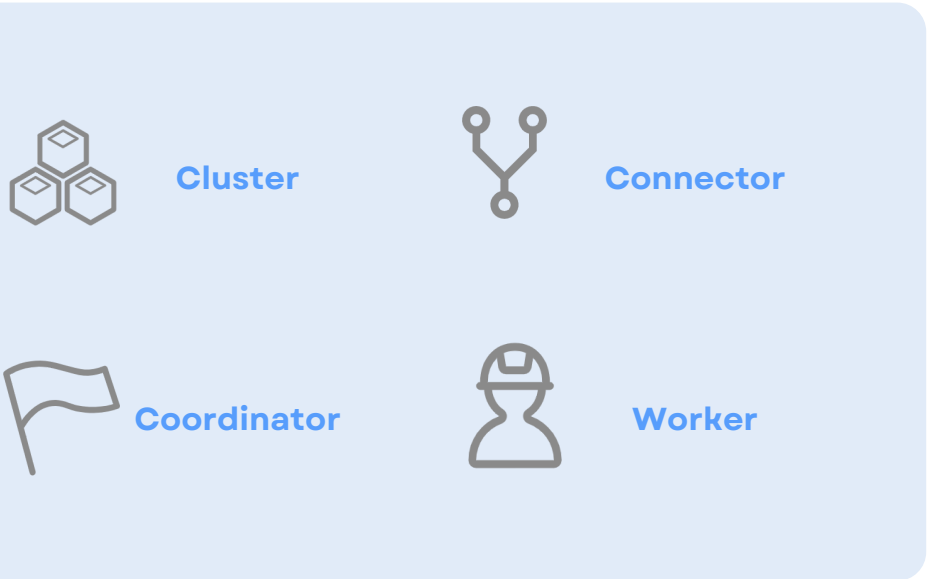
So, let's get started and unlock the full potential of Presto for your data analytics needs!

Table of Contents

CHAPTER

01 Presto Architecture and Key Concepts for Optimization

In this chapter, we will first provide an overview of Presto's high-level architectural components to help you understand how Presto works before diving into query optimization. Next, we will discuss the query lifecycle and execution model of Presto in detail, which is essential for diagnosing and tuning slow queries.



1.1 A Recap of Presto Architecture and Components

Presto is a distributed query engine that processes data in parallel across multiple servers, consisting of a single coordinator and multiple workers. Below is a brief recap of key components in Presto's architecture.

- **Cluster**: A Presto cluster comprises a coordinator and many workers. The coordinator manages query execution while the workers access data sources configured in catalogs and process each query in parallel.
- **Coordinator**: The coordinator is responsible for parsing SQL statements, planning queries, and managing Presto worker nodes. It creates a logical and physical plan of a query involving a series of stages and tasks running on a cluster of Presto workers.
- **Worker**: A Presto worker executes tasks and processes data, fetching data from data storage via connectors and exchanging intermediate data with other workers. The coordinator collects results from the workers and returns the final results to the client.
- **Connector**: A connector adapts Presto to a data source, like Hive data warehouse, data lake or even a relational database, allowing Presto to interact with a resource using a standard API. Presto contains dozens of built-in connectors, with many third-party developers contributing additional connectors for a variety of data sources.

1.2
Presto Query Lifecycle and How It Relates to Optimization

Understanding the query execution model is crucial for tuning Presto's performance. Let’s take a look at the Presto query life cycle.

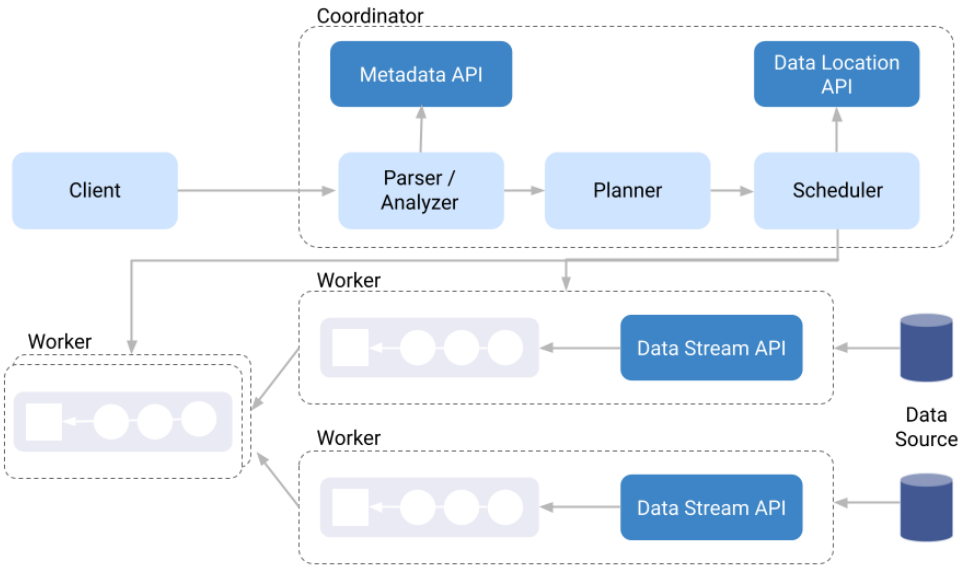


Figure 1. Presto : SQL on everything

1.2.1 Presto Coordinator

The Presto coordinator is responsible for parsing SQL statements, planning queries, and returning the results to the client. You can also configure the coordinator to act as a Presto worker to participate in SQL execution, which might be helpful in small Presto clusters.

When a SQL statement is submitted to the coordinator, it parses, analyzes, and creates a query plan. For common table layouts in analytical workloads, such as Hive, Iceberg, and Hudi tables, Presto obtains metadata and data statistics from third-party services or external storage. The coordinator uses this metadata and data statistics to generate the query plan and create a distributed query plan, facilitating parallel processing across Presto workers.

Metadata reflects the table layout, while data statistics reflect data distribution. Both significantly impact query plan generation, which in turn affects query performance. In this book, we will discuss optimizing table layout and gathering statistics to improve query performance.

A distributed query plan consists of one or more stages, each containing one or more tasks. Tasks process data in the form of splits, with the coordinator assigning tasks from a stage to Presto workers in the cluster. Specific operations on the data depend on the underlying data source and connector. Presto workers execute these operations and pass the results to other workers for further processing or return the final results to the coordinator, which then sends them to the client.

1.2.2 Presto Worker

The Presto worker is responsible for reading data from external storage, processing data using SQL operations, exchanging data with other Presto workers, and aggregating data to produce the final query results.

Presto is a storage-independent compute engine, meaning it always loads data from external storage. The size of data files and the ability to skip unnecessary data reads are essential in Presto. Additionally, using faster storage, better network bandwidth, locating storage closer to the Presto cluster, or using a cache can also speed up data reading. Subsequent chapters will discuss best practices for improving data access efficiency, such as using compression, partitioning tables, using columnar data formats like ORC and Parquet, and applying proper predicate pushdown.

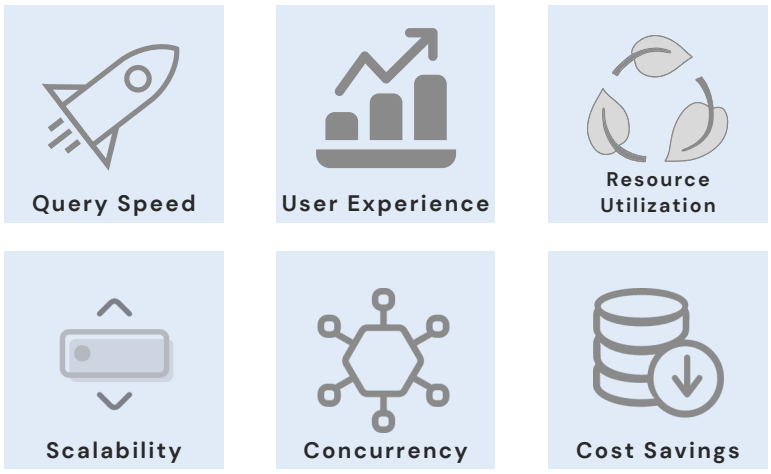
Presto workers provide computing power and are responsible for loading data for the Presto cluster. In most cases, equipping each worker with more and stronger CPUs will speed up queries. Furthermore, since Presto is storage-independent, faster access to external storage also speeds up query execution. For some operations, such as join, Presto needs to exchange data with other workers. In these cases, good network bandwidth is highly beneficial.

Memory is another crucial resource. By default, Presto uses memory instead of disks to store intermediate operation results. In daily operation and maintenance, you can consider the memory of all workers as a distributed memory resource pool. The pool's size determines the Presto cluster's processing and concurrency capabilities for queries. If a query's memory usage exceeds the total memory pool, the query will fail or be blocked indefinitely. When multiple queries run simultaneously, it's essential to balance the maximum memory usage per query and the maximum concurrency. We will discuss this in detail in later chapters.

1.3 The Importance of Optimization in Presto

Optimization plays a critical role in the performance and efficiency of any Presto deployment. Some key reasons why optimization is essential in Presto are:

- **Query performance**: Optimizing Presto allows for faster query execution, reducing the overall time taken to process queries. This can lead to significant productivity improvements, particularly when dealing with large-scale data processing tasks.
- **End-user experience:** Enhanced optimization results in a smoother end-user experience by ensuring faster response times and reducing query latency. This contributes to user satisfaction and enables more effective data analysis.
- **Resource utilization:** Efficient optimization enables Presto to use resources better, including CPU, memory, network and storage. This helps minimize the hardware footprint and associated costs while ensuring queries are processed as quickly as possible.
- **Scalability:** An optimized Presto deployment can handle increasing data volumes and query complexity better, ensuring that performance remains consistent as the system grows. This is especially important in today's data-driven world, where organizations continually process larger amounts of data and demand faster insights.
- **Concurrency:** Proper optimization can improve Presto's ability to handle concurrent queries, enabling multiple users to access the system simultaneously without experiencing performance degradation. This is crucial for maintaining a responsive and efficient environment, particularly in multi-user or multi-tenant deployments.
- **Cost savings:** By optimizing Presto, organizations can reduce infrastructure costs and save operation costs because of higher data engineer productivity. Efficient resource utilization and improved performance also contribute to these cost savings, helping organizations achieve a better return on their investment.



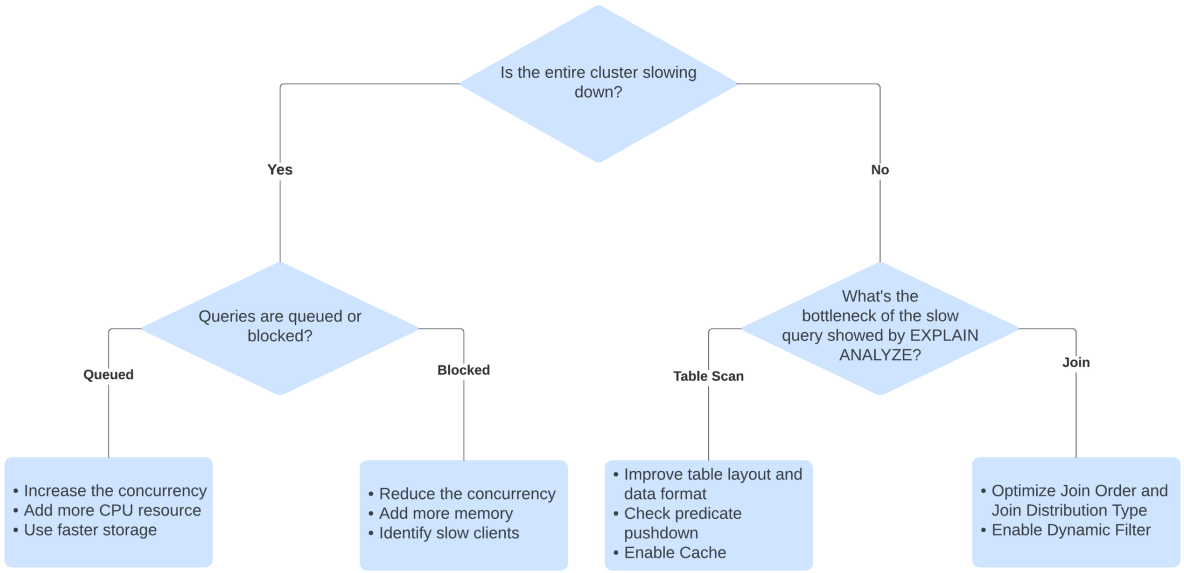
Chapter 1 Summary

In this chapter, you have learned the fundamentals of Presto's architecture and key concepts for optimization. This foundation is crucial for grasping the best practices and strategies presented in the next chapter.

CHAPTER

02 Best Practices for Presto Optimization

In this chapter, we will discuss various best practices and strategies for Presto performance tuning, ensuring optimal query performance and resource utilization.



The flow chart presented above outlines the recommended optimization process. Make sure that you structure your thoughts and follow these steps before digging into the specific tuning practices.

To begin, you need to determine whether the entire cluster is slowing down or if only a few queries are particularly slow.

If the entire cluster is slowing down, it often indicates that resources are insufficient or that resource allocation and maximum concurrency settings are unreasonable. Like many computing engines, Presto is resource-intensive. When resources are lacking, you will observe a decline in the cluster's overall throughput. In such cases, examine the Presto cluster's Web-UI to determine whether there are more blocked queries or more queries in the queue, so you can take appropriate measures.

On the other hand, if only a few specific queries are slow, you can identify the major bottlenecks of those queries using EXPLAIN ANALYZE results, and then optimize against those bottlenecks.

The following sections of this chapter will explore these optimizations and settings in greater detail.

2.1 Optimize Resource Allocation

Ensuring adequate memory allocation is crucial for query performance. It is important to monitor memory usage and adjust memory settings as necessary. Presto allows you to set the maximum memory per query and the maximum concurrent queries to control resource allocation.

Based on the total resource allocated to the Presto cluster, you will need to figure out a balance between maximum memory per query and the maximum number of concurrent queries. When adjusting the parameters for max concurrency and max memory, you may encounter the following common issues: too many blocked queries or too many queued queries.

If you notice a large number of blocked queries, you could try adding more memory. However, if your total memory resources are limited, consider reducing the maximum number of concurrent queries using Presto's resource group settings. Keep in mind that other factors may contribute to a high number of blocked queries, including, but not limited to:

- Memory allocation deadlock: consider adding query.low-memory-killer.policy to Presto's configuration
- Slow clients: some clients may struggle to retrieve results quickly enough

If you see many queued queries, try improving network or storage bandwidth, adding more resources, or increasing maximum concurrency to achieve a balance. Possible solutions include:

- Accelerating I/O or using faster storage if I/O bound
- Adding more CPUs if CPU bound

Presto Property Name	Suggested Value	Comments
query.max-memory-per-node	JVM heap size / max_concurrent	This value can fluctuate. A higher value can speed up large queries, and a lower value can reduce the risk of memory allocation deadlocks.
query.max-total-memory-per-node	1.5～2x of query.max-memory-per-node	
query.max-memory	Do not set this limitation if the query.max-memory-per-node has been set properly	
query.low-memory-killer.policy	total-reservation (Only set this value when you're suffering from memory allocation deadlock)	This value will kill the query currently using the most total memory.

After balancing max concurrency and max memory per query, you may need to improve the throughput of the entire Presto cluster by optimizing data storage and query execution. By monitoring and adjusting these settings as needed, you can ensure optimal resource allocation and query performance in your Presto environment.

2.2
Identify Bottlenecks Using EXPLAIN and EXPLAIN ANALYZE

To analyze query plans and performance, use the **EXPLAIN** and **EXPLAIN ANALYZE** commands. **EXPLAIN** shows the query plan, while **EXPLAIN ANALYZE** executes the query and provides additional information about the actual execution, such as the time taken and the number of rows processed.

```
EXPLAIN select * from customers limit 5;
```

OR

```
EXPLAIN ANALYZE select * from customers limit 5;
```

The results of **EXPLAIN** and **EXPLAIN ANALYZE** contain several fragments, as shown below:

```
Fragment 2 [SOURCE]
     CPU: 5.75s, Scheduled: 27.24s, Input: 135937 rows (12.21MB); per task: avg.:
45312.33 std.dev.: 42281.45, Output: 86 rows (8.85kB)
     Output layout: [c_customer_sk, c_customer_id, c_current_cdemo_sk,
c_current_hdemo_sk, c_current_addr_sk, c_first_shipto_date_sk, c_first_sales_date_
     Output partitioning: SINGLE []
     Stage Execution Strategy: UNGROUPED_EXECUTION
     - LimitPartial[5] => [c_customer_sk:integer, c_customer_id:char(16),
c_current_cdemo_sk:integer, c_current_hdemo_sk:integer, c_current_addr_sk:integ
     CPU: 2.00ms (0.03%), Scheduled: 20.00ms (0.07%), Output: 86 rows (8.85kB)
           Input avg.: 3988.68 rows, Input std.dev.: 120.47%
       - TableScan[TableHandle {connectorId='hive',
connectorHandle='HiveTableHandle{schemaName=tpcdssf1alluxio, tableName=customer,
analyzePartitionVa
         CPU: 5.93s (99.93%), Scheduled: 27.75s (99.87%), Output: 123649 rows (10.74MB)
         Input avg.: 4385.06 rows, Input std.dev.: 105.48%
         LAYOUT: tpcdssf1alluxio.customer{}
         c_customer_id := c_customer_id:char(16):1:REGULAR (1:31)
         c_email_address := c_email_address:char(50):16:REGULAR (1:31)
         c_current_addr_sk := c_current_addr_sk:int:4:REGULAR (1:31)
         c_birth_month := c_birth_month:int:12:REGULAR (1:31)
         c_current_cdemo_sk := c_current_cdemo_sk:int:2:REGULAR (1:31)
         c_customer_sk := c_customer_sk:int:0:REGULAR (1:31)
         c_last_name := c_last_name:char(30):9:REGULAR (1:31)
         c_salutation := c_salutation:char(10):7:REGULAR (1:31)
         c_last_review_date := c_last_review_date:char(10):17:REGULAR (1:31)
         c_first_shipto_date_sk := c_first_shipto_date_sk:int:5:REGULAR (1:31)
         c_current_hdemo_sk := c_current_hdemo_sk:int:3:REGULAR (1:31)
         c_birth_day := c_birth_day:int:11:REGULAR (1:31)
         c_preferred_cust_flag := c_preferred_cust_flag:char(1):10:REGULAR (1:31)
         c_first_sales_date_sk := c_first_sales_date_sk:int:6:REGULAR (1:31)
         c_login := c_login:char(13):15:REGULAR (1:31)
         c_first_name := c_first_name:char(20):8:REGULAR (1:31)
         c_birth_year := c_birth_year:int:13:REGULAR (1:31)
         c_birth_country := c_birth_country:varchar(20):14:REGULAR (1:31)
         Input: 135937 rows (12.21MB), Filtered: 9.04%
```

Compared to **EXPLAIN**, **EXPLAIN ANALYZE** offers actual execution statistics (highlighted in the example output above).

By examining the execution statistics of each step, you can identify bottlenecks within the query execution process.

In the example above, the actual CPU cost of Fragment 2 is 5.75s, with the TableScan (a sub-operator of Fragment 4) taking 5.93s, which constitutes the majority of CPU usage in Fragment 2. During performance tuning, you should focus more on the ScanFilterProject part, as it offers greater potential for performance gain.

Different kinds of queries often have different bottlenecks. The following two bottlenecks are most common:

- Table scan: In this case, consider implementing a better table layout, predicate pushdown, or using faster storage.
- Join: In this case, optimize the join distribution type and join order.

We will discuss specific optimization methods in subsequent sections.

2.3 Optimizing File Formats and Table Layouts

Choosing the best file format, compression, partitioning, bucketing, and sorting for your data can significantly impact query performance. Opt for formats like ORC or Parquet, which are columnar and support predicate pushdown. Use partitioning and bucketing to optimize data storage and retrieval, and employ appropriate compression algorithms to balance storage space and query speed.

ORC typically outperforms Parquet in Presto due to a longer history of optimization. However, Parquet has better adoption in the open-source community, and the Presto community is working to improve Parquet file performance.

In some scenarios, the data type of columns might significantly impact optimization. For example, if you use DOUBLE, REAL, and unorderable data types, the min-max dynamic filter won't work on these columns.

2.3.1 Columnar Data File Formats and Compression

When Presto reads a columnar data file, such as Parquet or ORC, it first reads the file metadata stored at the end of the file in the form of a footer. This metadata determines the structure of the data file and the location of each data section, such as a row group in a Parquet file. The metadata includes information like the schema of the data, the number of rows, the compression codec used, and the offset of each data section within the file.

After reading the metadata, Presto will read the data pages in parallel. It uses multiple threads to read data from different parts of the file simultaneously. The data values for a single column are stored and read together, allowing for more efficient processing and compression. Columnar formats are beneficial for most SQL queries since they don't touch all columns, making it much easier to skip unnecessary columns.

Columnar file formats are also more friendly with predicate pushdown to reduce the rows scanned. The metadata in each data file includes the statistics like the minimum and maximum value for each column. These statistics are collected at two levels: each Parquet file and each sub-section (row-group). When Presto pushes down a predicate to the table scan stage, it utilizes the Parquet file reader to scan the data file. If the query has a filter on a column (e.g., column1 > 5), the predicate pushdown helps prune files and row-groups based on the column's statistics.

In earlier versions of Presto, utilizing a flat table column layout in Parquet or ORC reduced the cost of querying nested columns. From Presto version 0.240 onwards, the optimization known as dereference pushdown can prune unnecessary data. However, some limitations persist for dereference pushdown. If, after using EXPLAIN ANALYZE, you do not observe the expected benefits from dereference pushdown, you can still employ a flat table column layout as an alternative to using nested columns.

Columnar data formats also support more efficient compression, which saves storage space and network bandwidth. In columnar formats, data is organized by column, where duplicate values frequently occur, such as date, state, or city. This data organization method enhances compression efficiency. It is highly recommended to convert data to columnar formats like Parquet or ORC.

2.3.2 Partitioning and Bucketing Strategies

Partitioning divides your table into parts based on the value of some columns (partition column or partition key). For Hive tables, each partition's data is stored in a separate folder. If the data scanned for queries with predicates is limited to partition columns, Presto will avoid accessing unrelated partitions, thereby saving time on file listing and reading operations.

However, excessive partitions in a single table could lead to performance decline during planning and increased storage pressure due to numerous folders and data files. For instance, using customer_id as the partition key may create millions of partitions when you have millions of customers. In such cases, consider employing bucketing, a straightforward form of hash partitioning. Bucket your tables based on one or more columns, utilizing a fixed number of hash buckets.

Here is an example of creating a table with partitioning and bucketing:

```
CREATE TABLE customers (
  customer_id bigint,
  created_day date
)
WITH (
  partitioned_by = ARRAY[created_day],
  bucketed_by = ARRAY['customer_id'],
  bucket_count = 100
)
```

In this example, the table is partitioned by the customer's created date and bucketed by the customer's ID, with a fixed total number of 100 buckets.

If you have many buckets, you may see a lot of empty buckets. You can eliminate them by setting the following session key:

```
SET SESSION create_empty_bucket_files = false;
```

2.3.3 Utilizing Materialized Views

A materialized view is a precomputed dataset generated from a query. Unlike regular views, this data is stored for future use. Since the data is already calculated, querying a materialized view becomes faster than executing the original query, especially if the computation is resource-intensive. Implementing materialized views can minimize data scanning, enhance query performance, and optimize resource utilization within your Presto environment.

We recommend using materialized views to speed up expensive aggregation, projection, and selective joins on large tables, especially those that run frequently.

The example below demonstrates how to create a materialized view to store active customers who have logged in within the last 7 days:

```
CREATE MATERIALIZED VIEW active_customers
AS
    SELECT customer_id, created_day
    FROM customers
    WHERE last_login_day >= CURRENT_DATE - INTERVAL '7' DAY;
```

You can manually refresh the materialized view with the following command:

```
REFRESH MATERIALIZED VIEW active_customers
```

The refresh command might be expensive because it re-executes the SQL query and writes the result to storage. We recommend storing materialized views in faster storage, such as Alluxio.

By considering file formats, compression, partitioning, bucketing, and materialized views, you can significantly enhance your Presto query efficiency and overall system performance by reducing the total amount of data scanned.

2.4 Data Caching for Presto

Using data cache in Presto is essential because the table scan operation can often become a bottleneck in query processing. Table scans involve reading large amounts of data, which can be a resource-intensive operation. Without cache, this process can consume significant amounts of compute cycles and network resources, resulting in slower query processing times. By utilizing cache, frequently accessed data can be stored in cache, reducing the need for repeated table scans.

Data caching is particularly advantageous for optimizing queries that access substantial volumes of data from remote data centers or clouds. By decreasing the number of remote table scans, cache substantially reduces query latency, saves on egress and cloud storage API costs, enhances resource utilization, and ultimately results in faster, more efficient query processing in Presto.

2.4.1 Types of Data Caching and Multi-tier Caching Architecture

Presto's data cache solution comprises two levels: the built-in cache and the Alluxio distributed cache, provided by a third-party solution.

The built-in cache encompasses the metastore cache, file list cache, and Alluxio SDK cache. It utilizes the memory and SSD resources of the Presto cluster, operating within the same process as Presto for optimal performance without network communication overhead. However, due to resource competition within the Presto cluster, the built-in cache's capacity is limited.

Conversely, third-party cache, such as Alluxio distributed cache is independently deployable, and offers better scalability and increased cache capacity, proving particularly advantageous for large-scale analytics workloads.

The table below presents different cache types, their corresponding resource types, locations, and recommended use cases. It is important to note that none of these caches are enabled by default in Presto; modifications to Presto's configuration are required for activation.

Type of Cache		Cache Location	Storage Media	When to Use
Built-in Presto Cache	Metastore Cache	Presto Coordinator	Memory	• Slow planning time • Slow Hive Metastore • Large tables with hundreds of partitions
	List File Cache	Presto Coordinator	Memory	• Overloaded HDFS's namenode • Overloaded object store such as S3
	Alluxio SDK Cache	Presto Workers	Memory/SSD	• Slow or unstable external storage
Third-party Cache	Alluxio Distributed Cache	Alluxio Workers	Memory/SSD/ HDD	• Cross-region, multi-cloud, hybrid-cloud • Data sharing with other compute engines

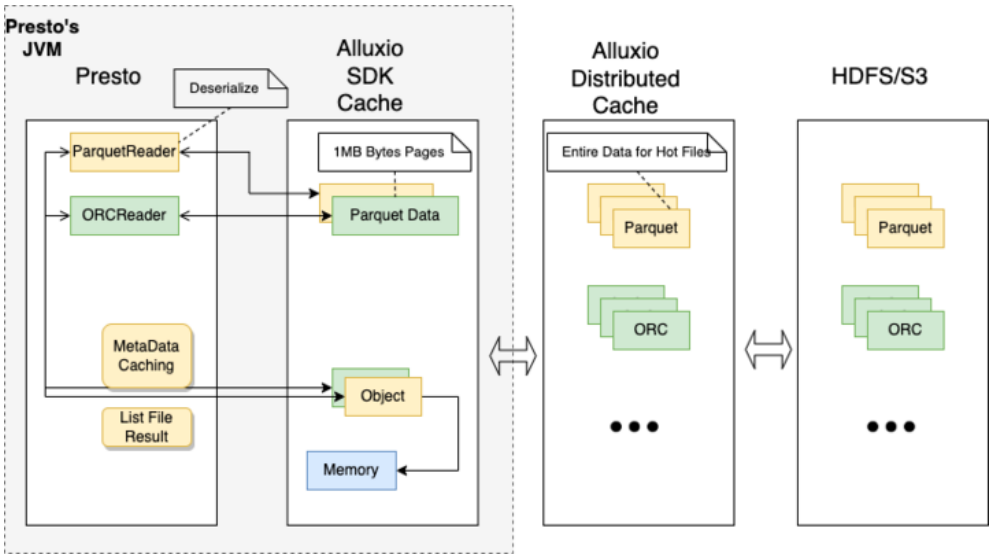
When encountering slow planning times or an overloaded Hive Metastore, using the Metastore cache in Presto effectively reduces planning time and the number of requests sent to Hive Metastore. This is especially helpful with large tables containing hundreds of partitions or more. The Metastore cache stores partition metadata, enabling quicker access and minimizing repeated Hive Metastore queries.

In situations where the HDFS namenode is overloaded or when using object stores with poor file listing performance, the list file cache substantially improves query latency. This cache stores file paths and their attributes, reducing the need to repeatedly retrieve this information from the namenode or object store.

For slow or unstable external storage, the Alluxio SDK cache significantly enhances query performance by caching data in memory and reducing repeated data transfers.

Note that all built-in caches are resource-intensive. If the Presto coordinator or worker's memory or storage resources are insufficient for large datasets, setting up an Alluxio distributed cache cluster provides expansive cache space for faster access to frequently accessed data.

Integrating various cache types results in a multi-level cache architecture and data access strategy within Presto, as shown in the architecture diagram below. When accessing data, Presto first checks the built-in cache. If there is a hit, data is accessed from the cache; otherwise, Presto searches the Alluxio distributed cache (when deployed). If both caches miss, Presto retrieves data from the underlying storage. This strategy significantly reduces data access latency, especially for remotely stored data. In addition to latency reduction, a caching system can also decrease costs, including egress fees and object storage API call expenses. By reducing the frequency of data access to underlying storage, caching improves cost-efficiency and overall effectiveness of the data processing pipeline.



2.4.2 Built-in Caching in Presto

Presto's built-in caching solutions, including metadata caching and Alluxio SDK cache, significantly enhance the performance and efficiency of Presto clusters. Metadata caching reduces query planning latency, while Alluxio SDK cache minimizes latency in table scans. These caching mechanisms enable faster access to frequently accessed data and decrease repeated network requests, improving overall performance.

By employing these caching solutions, you can anticipate quicker query response times, reduced network overhead, and a more efficient process for API calls to data source storage and Hive Metastore. Furthermore, these caching mechanisms can lead to cost savings by decreasing the number of API calls needed to access remote data and by minimizing egress fees related to data transfers across regions.

It is essential to note that most caching mechanisms require memory resources from the Presto Coordinator. If the Coordinator stores too many objects in the cache, it may encounter JVM GC (garbage collection) issues. In extreme cases, the Coordinator may experience an OOM (Out of Memory) failure and terminate unexpectedly.

To prevent these issues, it is advised to enable each caching solution individually and closely monitor the Presto Coordinator's memory usage. By doing so, you can ensure optimal performance and stability of your Presto platform.

Metastore Cache

Presto's metastore cache stores results from the Hive Metastore, which can be very useful when the Hive Metastore is overloaded with requests or becomes overwhelmed. Caching metadata in memory allows for faster query execution and reduced overhead, decreasing the overall load on the Hive Metastore. To enable metadata caching for Hive catalogs, use the following settings:

```
hive.partition-versioning-enabled=true
hive.metastore-cache-scope=ALL
hive.metastore-cache-ttl=1d
hive.metastore-refresh-interval=1d
hive.metastore-cache-maximum-size=10000000
```

Keep in mind that if tables are frequently updated, configuring a shorter TTL or refresh interval for the Metastore versioned cache helps prevent Presto queries from reading stale data. By setting a shorter interval for refreshing the cache, you can ensure that only the most current metadata is stored, reducing the likelihood of outdated metadata being used in query execution.

List File Status Cache

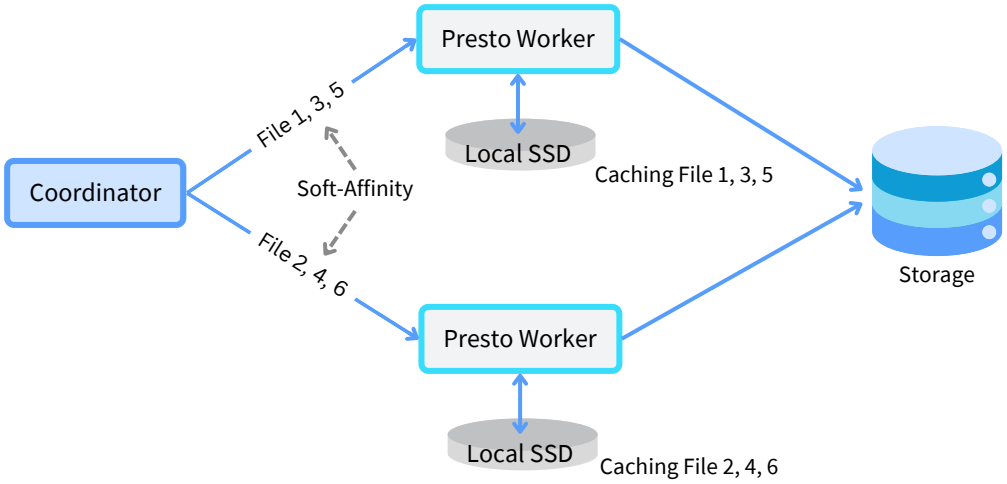
List file calls can be a significant bottleneck for storage engines like HDFS and S3, potentially overwhelming the name node in the case of HDFS and increasing the cost of S3 storage. To address this issue, the Presto coordinator caches file lists in memory, allowing for faster access to frequently used data and reducing the need for lengthy listFile calls to remote storage. However, it is worth noting that this caching mechanism can only be applied to sealed directories, as Presto skips caching open partitions to ensure data freshness.

To enable list file status caching, use the following settings:

```
hive.file-status-cache-expire-time=1h
hive.file-status-cache-size=10000000
hive.file-status-cache-tables=*
```

Alluxio SDK Cache

Alluxio SDK cache is a built-in caching solution designed to reduce latency associated with table scans. By caching data on Presto worker's local SSD, Presto can quickly access and query the data, minimizing repeated network requests and enhancing query performance.



Enable the Alluxio SDK cache with the settings below:

```
hcache.enabled=true
cache.type=ALLUXIO
cache.base-directory=file:///tmp/alluxio
cache.alluxio.max-cache-size=100MB
```

To achieve the best cache hit rate, you may need to change the node selection strategy to soft-affinity:

```
hive.node-selection-strategy=SOFT_AFFINITY
```

Soft-affinity refers to a scheduling mechanism that attempts to send requests to specific workers based on file paths. The goal is to maximize the cache hit rate by ensuring the data needed for a query is already present in the worker's cache.

Soft-affinity is called "soft" because it's not a strict rule that the split must be sent to a particular worker. If the preferred worker is too busy, the scheduler will send the split to another available worker instead of waiting for the preferred worker to become available.

If you encounter errors such as "Unsupported Under FileSystem," download the latest Alluxio client JAR from the Maven repository (https://mvnrepository.com/artifact/org.alluxio/alluxio-shaded-client) and place it in the {$presto_root_path}/plugin/hive-hadoop2/ directory.

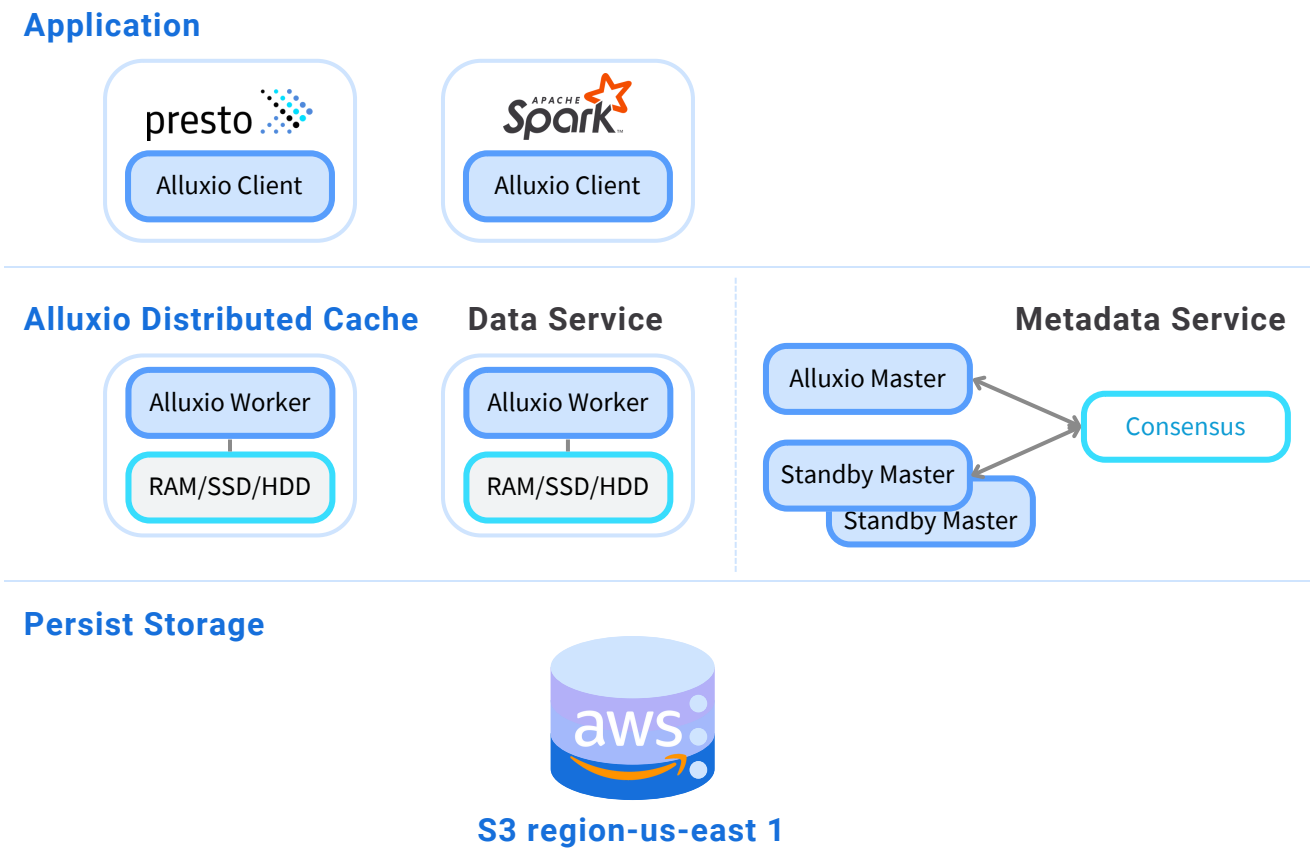
Benefits of using Alluxio SDK cache include faster query response times, improved query performance, and reduced network overhead. By minimizing repeated network requests, Alluxio SDK cache can also help save on egress fees and cloud storage API call costs associated with accessing remote data.

2.4.3 Third-party Caching (Alluxio Distributed Cache)

Utilizing a third-party distributed caching layer can offer several optimizations to Presto queries:

- Enhancing query performance by reducing I/O access latency while co-locating with Presto
- Accelerating queries when reading remote data across datacenters, regions, and clouds
- Serving as a shared cache between Presto workers, across Presto clusters, and other compute engines like Apache Spark
- Providing resilient caching, allowing you to save money using spot instances without losing cached data

Alluxio provides a distributed caching layer that can be used between Presto and data sources to improve I/O performance. By caching data closer to the Presto workers, Alluxio reduces the latency of data access and alleviates pressure on the underlying storage system. The architecture diagram below illustrates how Alluxio caching works.



Alluxio distributed cache protects against additional costs and performance degradation, particularly when using S3 and underlying storage. This is because Presto utilizes the HTTPS protocol to fetch data files stored in your S3 bucket. As mentioned previously, when predicate pushdown is effective for columnar data formats like Parquet and ORC, it skips numerous data pieces in the file, potentially causing read fragmentation. This fragmentation increases the

number of S3 API calls by 100 to 1000 times. However, by placing Alluxio between Presto and S3, all read requests will be directed to Alluxio. With an HDFS-compatible client and a buffer, Alluxio mitigates the impact of read fragmentation.

To get started with Alluxio distributed cache for Presto, follow this 5-minute tutorial (sandbox): https://www.alluxio.io/blog/getting-started-with-the-alluxio-presto-sandbox/

2.5 Gathering and Using Hive Table Statistics

Collecting and updating statistics for Hive tables is crucial for helping the Presto optimizer make better decisions when planning queries. You can use the following methods to gather and maintain Hive table statistics, ensuring that the optimizer has the most accurate information.

To display the current statistics of a table, use the **SHOW STATS** command:

```
SHOW STATS table_name
```

To gather statistics for a table, use the **ANALYZE** command. Update these statistics periodically to maintain accurate information. It might take a while to recalculate and generate the stats for the entire table, which is why it is advised to generate stats for only one or two columns later on.

```
ANALYZE table_name
```

If you want to refresh the statistics for specific partitions within a Hive table, specify the partition key values (columns) to collect stats for those partitions:

```
ANALYZE table_name WITH (
    partitions = ARRAY[
        ARRAY['Dec', '1'], ARRAY['Dec', '2']])
```

The command above will refresh the stats for the two partitions 'Dec/1' and 'Dec/2' (assuming the table is partitioned by month and day).

For tables with a large number of columns, gathering statistics for all columns can be resource-intensive. In such cases, you can choose to collect statistics only for a subset of columns. We recommend focusing on join keys, grouping keys, and filtering keys

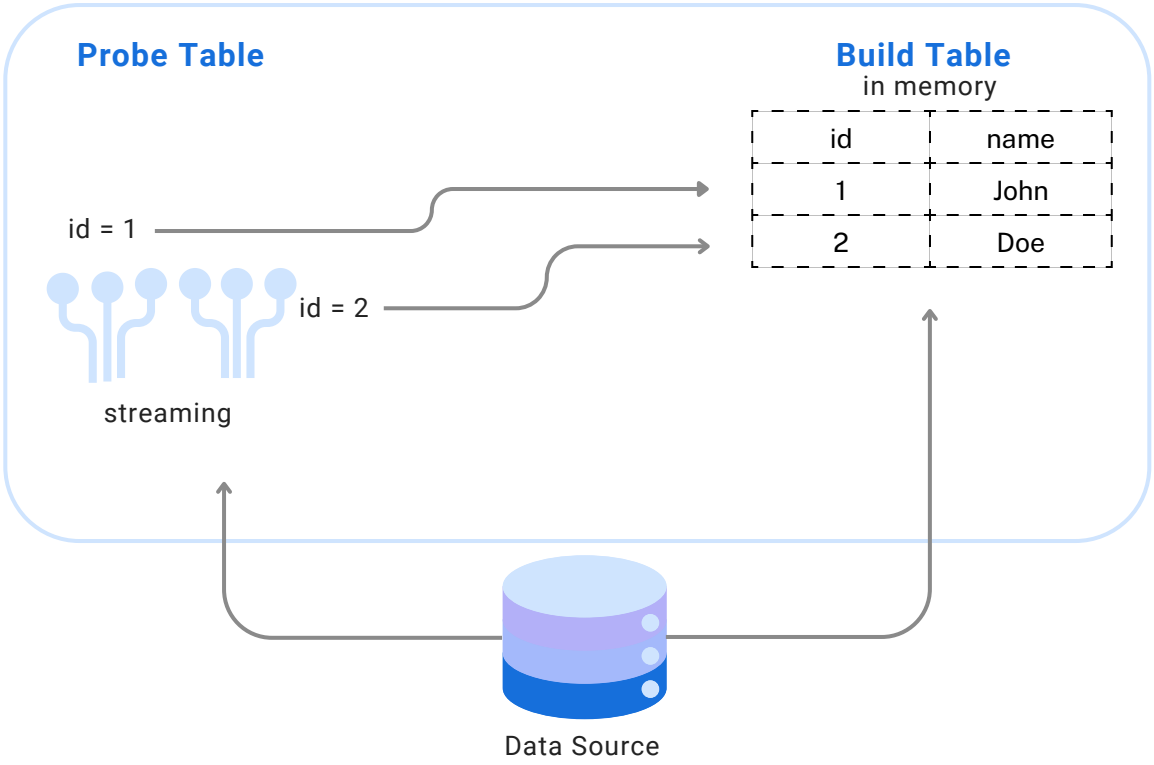
```
ANALYZE table_name WITH (
    columns = ARRAY['column1', 'column2'])
```

Maintaining accurate statistics can significantly improve query performance. Presto can optimize SQL planning based on these stats, leading to better join orders and join types for queries. This optimization can substantially reduce the amount of data read and exchanged by Presto workers, resulting in more efficient query execution.

2.6 Join Optimization Strategies

Optimizing join operations can significantly improve query performance. By considering join distribution types and join orders, you can use Presto's cost-based optimizer to determine the best join strategy based on table statistics.

Presto always uses a hash join algorithm in its implementation. It is more efficient for Presto to read one table into memory once and reuse its values while scanning the data of the other table. The table read into memory is called the "build table," and the other table is called the "probe table."



The size of each table is important when fitting one of them into memory. To save memory and improve the maximum concurrency of the Presto cluster, it is preferable to choose the smaller table as the build table. This is why table statistics are important for join optimization.

2.6.1 Types of Join Distributions

Presto uses a hash-based join algorithm, which means each Presto worker will build a hash table from the build table. Then, Presto will stream the probe table, and for each row, it will look up the hash table to find matching rows.

There are two types of join distributions:

- Partitioned: Each node builds a hash table from only a fraction of the data of the build table.
- Broadcast: Each node builds a hash table from all of the data, meaning the build table data is replicated to each node.

Broadcast join can be faster but requires the build table data to fit in the memory of the Presto worker. In this case, join order is crucial, and the smaller table should be selected as the build table.

Partitioned join can be slower as it requires redistributing both tables using a hash of the join key. However, the memory requirement of each node can be lower, as the build table data only needs to fit in the total memory among all Presto workers.

By default, Presto automatically chooses whether to use a partitioned or broadcast join and selects the probe and build sides.

You can change the join distribution strategy by setting the join_distribution_type session property:

```
SET SESSION join_distribution_type = AUTOMATIC
```

Possible values are:

AUTOMATIC	Join distribution type is determined automatically for each join
BROADCAST	Broadcast join distribution is used for all joins
PARTITIONED	Partitioned join distribution is used for all joins

You may want to adjust the maximum memory Presto workers can use to build the hash table for broadcast joins:

```
SET SESSION join_max_broadcast_table_size = 100M
```

A higher value might improve performance because a larger build table can also use broadcast join, but a lower value can improve the maximum concurrency of the Presto cluster.

2.6.2 Optimizing Join Orders

A proper join order can significantly reduce the amount of data read from external storage and the amount of data exchanged among workers.

By default, Presto uses cost-based join enumeration, meaning it uses table statistics (collected in the previous section) to estimate the costs for different join orders and automatically picks the join order with the lowest computed costs.

You can also specify the join reordering strategy using the following session key:

```
SET SESSION join_reordering_strategy = AUTOMATIC
```

The default value, "AUTOMATIC," automatically reorders the join based on the computed cost.

Other choices for join reordering strategy are:

`ELIMINATE_CROSS_JOINS`	Eliminate unnecessary cross joins
`NONE`	Purely syntactic join order

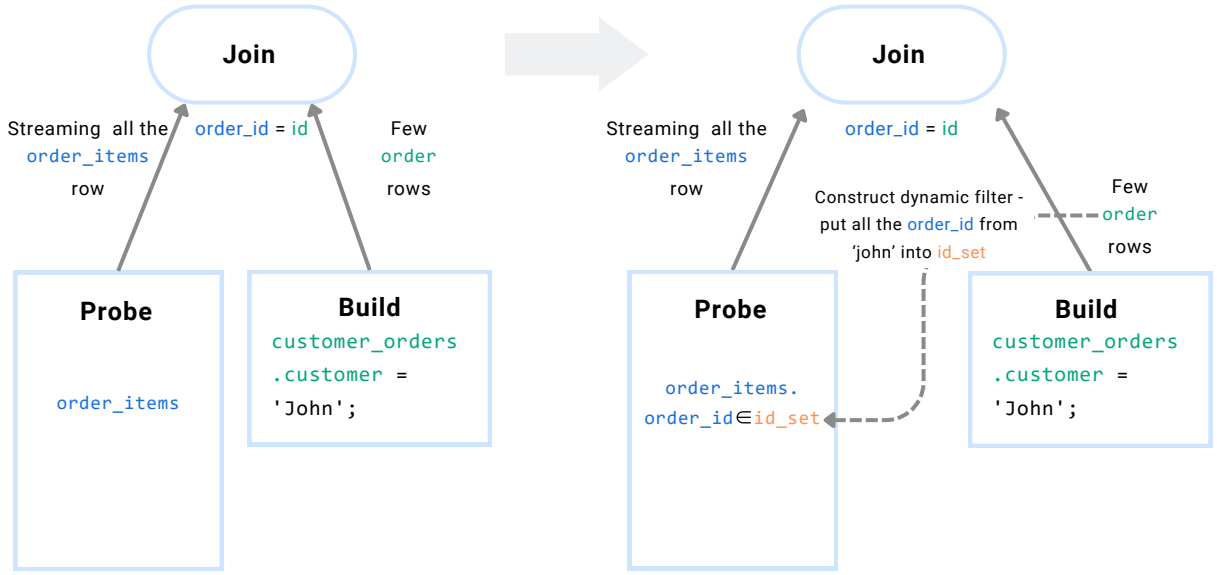
ELIMINATE_CROSS_JOINS reorders joins to eliminate cross joins, where possible, and otherwise maintains the original query order.

NONE maintains the order the tables are listed in the query.

2.6.3 Implementing Dynamic Filtering

In highly-selective join scenarios (common in ad hoc and dashboard use cases), most rows read from the probe table are not needed and will be ignored during the join, since they don't match the join criteria.

```
SELECT * FROM order_items JOIN customer_orders
ON order_items.order_id = customer_orders.id
WHERE customer_orders.customer = 'John';
```



When Presto uses a broadcast join, each worker collects values from the build table, and after its input is over, applies a dynamic filter generated by the build side to the table scan of the probe side. This dynamic filter can significantly reduce the rows read on the probe side.

By doing so, dynamic filtering can help improve query performance by reducing the amount of data processed during join operations.

Dynamic filtering is not enabled by default. To activate it, request that your Presto administrator add the following configuration property:

```
experimental.enable-dynamic-filtering = true
```

Alternatively, modify it using the session key:

```
SET SESSION jenable_dynamic_filtering = true
```

By implementing these join optimization strategies, you can significantly improve the efficiency and performance of your Presto queries. Remember to consider join distribution types and join orders while using Presto's cost-based optimizer, and to leverage dynamic filtering when appropriate.

2.7 Recommended JVM Configuration Settings

Tuning the JVM configuration settings can help improve Presto's performance and resource utilization. Adjust settings such as heap size, garbage collection options, and JVM flags to optimize Presto for your specific environment and workload.

Applying these best practices and tuning strategies can significantly improve the performance and efficiency of your Presto queries. Experiment with different settings and optimizations to find the best configuration for your use case.

JVM argument name	Suggested Value	Comments
-Xmx	70% ~ 85% of the physical memory of the server.	Highly recommended if Presto is the only major process running on the server
-XX:InitialRAMPercentage=	80.0	The initial heap size of JVM will be bound to 80% of your physical memory
-XX:MaxRAMPercentage=	80.0	The maximum heap size of JVM will be bound to 80% of your physical memory
-XX:-G1UsePreventiveGC		Disable the preventive GC to prevent too many objects from being promoted to the old gen for better GC efficiency

You will not need both -Xmx and MaxRAMPercentage in the JVM config, but mostly using MaxRAMPercentage could prevent your Presto process from getting killed by the OOM killer of the operating systems, especially when you are running Presto in a container.

It is not recommended to assign all the physical memory to Presto because some memory should be reserved for the operating system. Additionally, some dependencies, such as the Alluxio client, may use off-heap memory to improve performance. Note that off-heap memory usage won't be counted in the heap usage of JVM.

Chapter 2 Summary

In this chapter, you have learned the tuning steps and seven best practices for Presto optimization. By following these best practices and tuning strategies, you can significantly improve the performance and efficiency of your Presto queries, allowing you to tailor configurations for your specific environment and workload.

CHAPTER

03 Uber's Presto Optimization

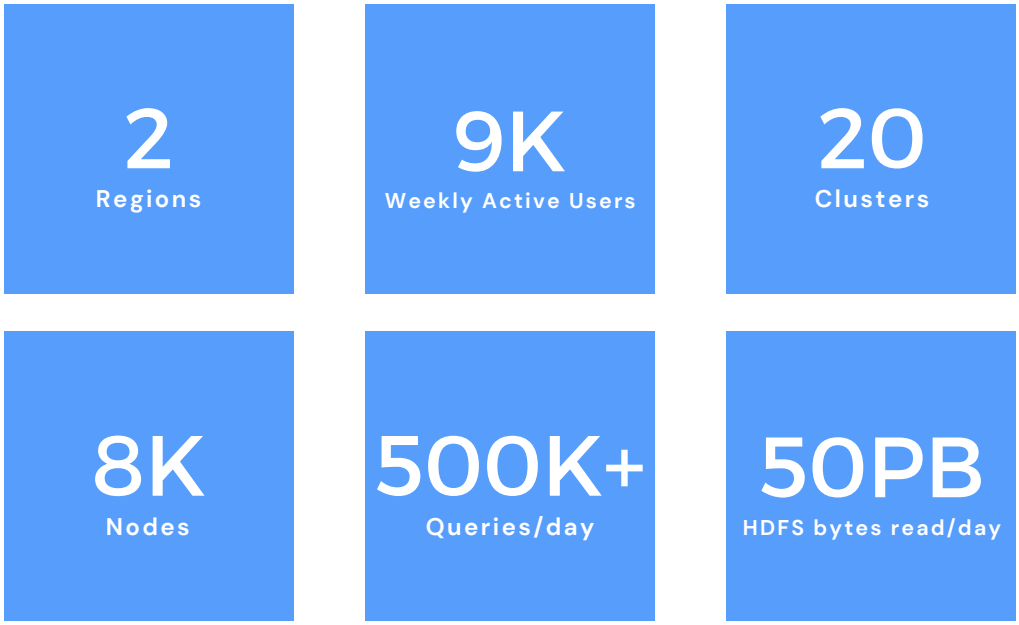
In this chapter, you will learn about real-world case studies that showcase the successful implementation of Presto optimization using the Alluxio SDK. Chen Liang, a senior software engineer from the Uber Presto team, shares how they have created a new caching layer for hot data needed by the Presto fleet based on recent open-source collaboration between Presto and Alluxio communities at Uber. This architecturally simple and clean approach has significantly reduced HDFS latency through managed SSD and consistent hashing-based soft affinity scheduling.

3.1 Background and Performance Challenges

Data informs every decision at Uber. Business lines heavily rely on data analytics to gain insights into operations, pricing, and customer behaviors. As a result, having an analytics platform with fast access to data is fundamental to all aspects of Uber's business.

Presto serves as the distributed query engine supporting data analytics, such as dashboarding at Uber. For instance, the operations team heavily relies on Presto for dashboarding, while UberEats and the marketing team base their pricing decisions on ad-hoc query results. By offering interactive queries with consistent performance, the platform can reduce time to insight and greatly benefit the business.

Uber operates a large-scale Presto system with the following scale:

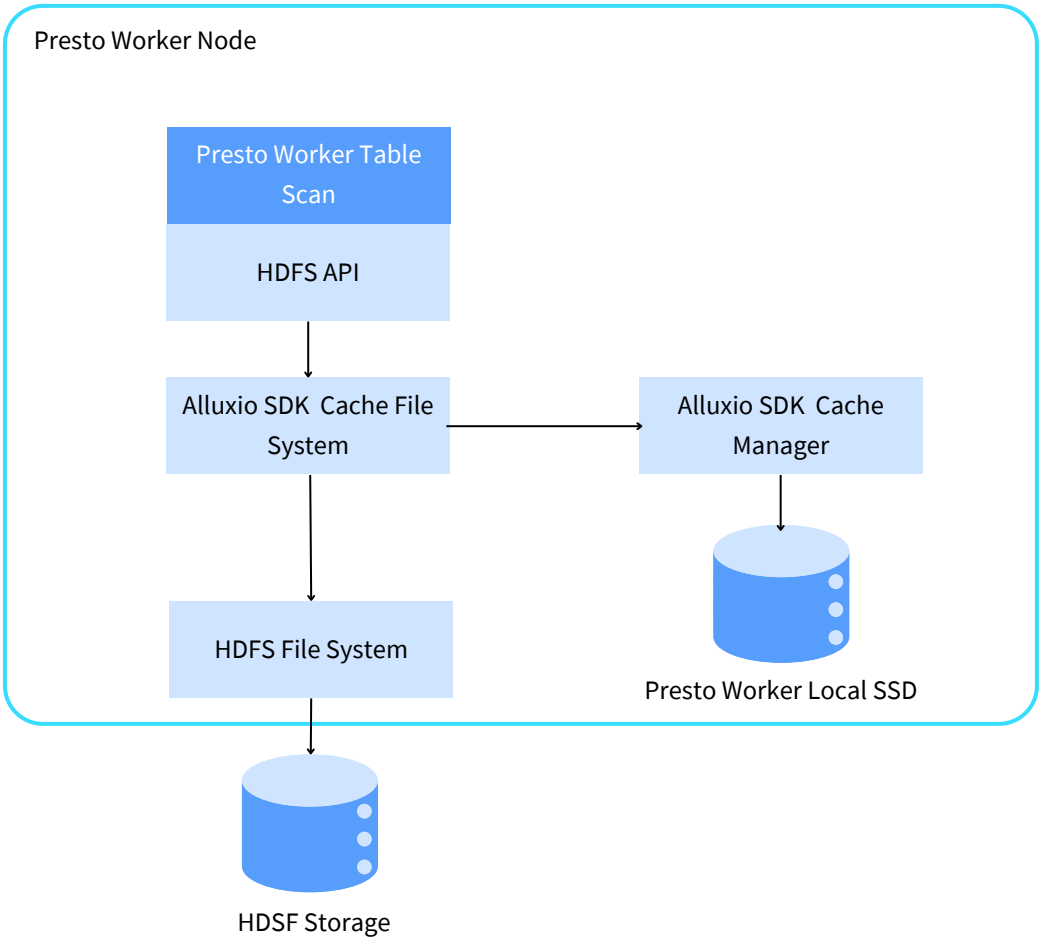


As data volume and analytics use cases increase, maintaining stable and low-latency query performance becomes challenging. The HDFS cluster is overloaded, leading to inconsistent query latency. A typical solution to overcome latency is caching hot data to eliminate repeated disk I/O. The challenge lies in designing and implementing a caching solution that addresses data access patterns and fits the architecture.

3.2 Architecture with Alluxio SDK Cache

The Uber Presto team has chosen the Alluxio SDK cache as their solution. Alluxio is integrated as a local library, leveraging Presto workers' local NVMe disks.

The architecture using the Alluxio SDK cache includes a local cache running inside the Presto worker. Presto and Alluxio SDK are implemented as a layer on top of the default HDFS client implementation. When any external API reads from an HDFS call, the system first checks the cache inventory to determine if it is a cache hit or miss. If it is a cache hit, the data is read directly from the local SSD. Otherwise, the data is read from the remote HDFS and cached locally for the next read.



3.3 Presto Optimization at Uber

3.3.1 Cache Filter

The cache hit rate greatly affects overall performance. Thus, Uber has implemented cache filtering to store only a selected subset of data, including specific tables and a limited number of partitions. The cache filter has boosted the cache hit rate from 65% to 90%.

3.3.2 Node ID-based Consistent Hashing

Consistent hashing minimizes cache hit rate drops when worker nodes join or leave. Instead of using a mod-based function, all nodes are positioned on a virtual ring. Maintaining the relative order of nodes on the ring reduces the number of cache keys needing remapping, regardless of nodes joining or departing.

3.3.3 Metadata Optimization

Introducing file-level metadata leads to multiple data versions. When data is updated, a new timestamp is generated, corresponding to a new version. A folder is created to store the new page, associated with this timestamp. Simultaneously, efforts are made to remove the old timestamp.

For more optimization details, please refer to this blog: https://www.uber.com/blog/speed-up-presto-with-alluxio-local-cache/

3.4 Results: Consistent Performance, Reduced HDFS Load, Faster Insights

The Uber Presto team deployed Alluxio SDK cache across three production clusters, totaling 1,500 nodes, each with NVMe disks and 1TB cache space per worker node. Below are the results:

- **Stable query performance with decreased variance:** By using Alluxio SDK cache, Uber has managed to stabilize the performance of their Presto queries. The Alluxio SDK cache has significantly reduced HDFS latency and minimized the variance in query times, providing a more consistent query performance.
- **Reduce 10% HDFS storage load:** With Alluxio SDK cache in place, Uber observed a 10% reduction in data read traffic to their HDFS cluster. If migrating to cloud storage, this reduction will lead to decreased storage API calls, helping lower Uber's future cloud storage costs.
- **Reduce the latency of input read by about 50%**. For tables cached in Alluxio SDK, the input latency during the ScanFilterProjectOperator stage was reduced by about 50%, enabling queries with heavy table scans to return results more quickly.
- **Faster insights for ad-hoc queries:** Because of the reduced latency, the Presto platform now provides faster insights for ad-hoc queries. This enables Uber's teams to make more informed decisions in a shorter time frame, enhancing overall business efficiency.

> Alluxio SDK cache has significantly improved the performance, efficiency, and scalability of our large-scale Presto deployments. This collaboration between Uber and the Alluxio open-source community has led to valuable optimizations in the Presto project, inspiring other organizations to adopt the Alluxio SDK cache and driving innovation in data analytics.
>
> – Chen Liang, Senior Software Engineer @ Uber

CHAPTER

04 Presto on Hybrid Cloud at Walmart

4.1 Data Access Challenges in Hybrid Cloud

Hybrid cloud architecture offers organizations the ability to harness the power of cloud computing for enhanced scalability and analytics while maintaining data storage on-premises. The data platform team at WalmartLabs aims to use hybrid cloud to expand their compute capacity and achieve more flexible resource allocation.

WalmartLab operates large-scale on-premises Hadoop clusters that support various domains, such as finance and supply chain management. Their data platform team has been working to load data into Google Cloud Platform (GCP), creating a parallel environment to their on-premises infrastructure. By bursting to the cloud while keeping data on-premises, WalmartLabs hopes to leverage additional compute capacity from public cloud providers.

However, hybrid cloud architecture introduces several challenges related to data access. One major issue is Presto query performance bottlenecks, which can arise due to unpredictable network I/O. Additionally, query patterns and datasets modeled in star schemas could potentially benefit from dimension table caching to further enhance performance.

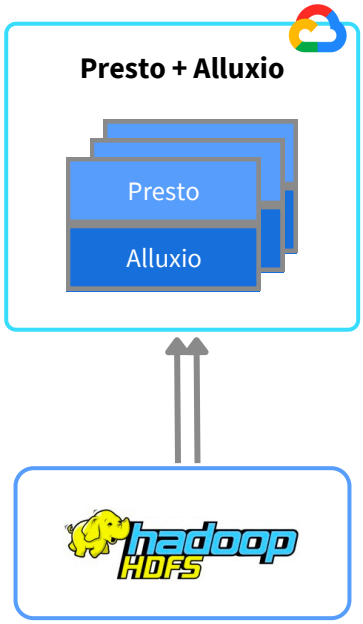
In addressing these challenges, WalmartLabs has outlined several requirements for their hybrid cloud analytics. Their primary goal is to accelerate Hadoop-based queries while achieving better scalability and effective cluster utilization through auto-scaling. Moreover, they seek to improve query performance without incurring network hops on recurrent queries, while simultaneously reducing costs by avoiding the need to persist data in the cloud or create duplicate data copies.

4.2 Architecture with Alluxio Distributed Cache on Hybrid Cloud

To meet the above requirements, WalmartLabs' data platform team selected Alluxio distributed cache to enable hybrid cloud data access. This caching solution facilitates the optimization of performance without creating data copies on the public cloud, while ensuring data remains accessible and secure.

The hybrid cloud architecture consists of Presto and Alluxio distributed cache in GCP and data stored in on-premises HDFS:

- Alluxio is co-located with Presto, ensuring data locality and minimizing the impact of data movement on query performance.
- Automatic metadata synchronization creates Hive tables with Alluxio mount points, streamlining the integration of Alluxio with WalmartLabs' existing data infrastructure.
- Frequently used data is pinned within Alluxio to prevent cache evictions, further enhancing performance and reducing latency.



4.3
Results: 2X Performance, Lower Cost, Higher Throughput

- **Consistent query latency and avoiding unpredictable network behavior:** The hybrid cloud architecture with Alluxio distributed cache offers consistent query performance. The frequently used tables are pinned in the cache on the public cloud, saving network bandwidth and circumventing unpredictable data transfers and minimizing the impact of network variability.
- **Performance improvements for range queries and higher throughput:** The hybrid cloud architecture with Alluxio distributed cache provides 2x faster results and more efficient querying process for range queries. The lower response times and improved concurrency also leads to higher throughput, allowing for more queries to be processed in a shorter time.
- **Cost reduction with half the compute costs or 2x compute capacity for the same environment:** The hybrid cloud architecture with Alluxio distributed cache enables WalmartLabs to handle growing data needs while maintaining consistent query performance. This optimization translates to either halving compute costs or doubling compute capacity within the same environment, ensuring that data platforms are prepared to meet the demands of the business.

For more information about Presto and Alluxio at Walmart, check out this presentation: https://www.alluxio.io/resources/videos/enterprise-distributed-query-service-powered-by-presto-alluxio-across-clouds-at-walmartlabs/

CHAPTER

05 Key Takeaways and Resources for Further Learning

5.1 Key Takeaways

Throughout this ebook, you have learned various best practices and strategies for optimizing Presto performance, focusing on identifying bottlenecks, memory management, file formats, table layouts, join optimization, dynamic filtering, and JVM configuration. You have also explored how to use caching to improve query performance in real-world case studies from Uber and WalmartLabs.

As we conclude, let us recap the best practices

- Optimize resource allocation by adjusting memory and CPU settings to maximize resource utilization and minimize contention.
- Identify bottlenecks using EXPLAIN and EXPLAIN ANALYZE to understand query performance and discover areas for improvement.
- Optimize file formats and table layouts by choosing columnar data file formats and compression, implementing partitioning and bucketing strategies, and utilizing materialized views for faster query execution.
- Leverage data caching for Presto, such as using Alluxio SDK cache and Alluxio distributed cache, to accelerate queries, reduce latency, and enable efficient data sharing among Presto workers and other compute engines.
- Gather and use Hive table statistics to help Presto make better decisions during query planning and execution.
- Implement join optimization strategies by understanding the types of join distribution, optimizing join orders, and employing dynamic filtering to improve query performance.
- Configure recommended JVM settings to enhance Presto's performance and resource utilization by adjusting heap size, garbage collection options, and JVM flags.

By leveraging the best practices, strategies, and resources outlined in this ebook, you can achieve optimal query performance and resource utilization.

5.2 Additional Resources

To further enhance your understanding and implementation of the techniques provided by this eBook, you can refer to the following resources:

- Dive deeper into Presto's features, configuration options, and performance tuning recommendations by visiting Presto's documentation
- Unlocking the full potential of Presto and Alluxio in your data analytics infrastructure:
  - Learn more about the synergy between Alluxio and Presto
  - Deploy Alluxio distributed cache with Presto by following Alluxio’s documentation - Running Presto with Alluxio
  - Attend online talks to learn from experts and share your experiences with the community. Alluxio webinars and open office hours
  - Join 10k+ members in the Alluxio community slack channel to ask any questions and provide your feedback.

References

[1] R. Sethi, M. Traverso, D. Sundstrom, D. Phillips, W. Xie, Y. Sun, N. Yegitbasi, H. Jin, E. Hwang, N.Shingte et al., “Presto: SQLon everything,” in 2019 IEEE 35th International Conference on Data Engineering (ICDE). IEEE, 2019, pp. 1802–1813.
[2] Presto official repository @ Github: https://github.com/prestodb/presto
[3] Prestodb Blog | RaptorX: Building a 10X Faster Presto: https://prestodb.io/blog/2021/02/04/raptorx
[4] Alluxio Blog | Speed Up Uber’s Presto with Alluxio | A collaboration between Uber and Alluxio – part 1: https://www.alluxio.io/blog/speed-up-ubers-presto-with-alluxio-part-1/
[5] Alluxio Blog | Designing the Presto Local Cache at Uber | A collaboration between Uber and Alluxio – part 2: https://www.alluxio.io/blog/designing-presto-local-cache-at-uber-part-2/
[6] Uber Engineering Blog | Speed Up Presto at Uber with Alluxio Local Cache: https://www.uber.com/blog/speed-up-presto-with-alluxio-local-cache/
[7] Alluxio | Enterprise Distributed Query Service Powered by Presto & Alluxio Across Clouds at WalmartLabs: https://www.alluxio.io/resources/videos/enterprise-distributed-query-service-powered-by-presto-alluxio-across-clouds-at-walmartlabs/

Chapter Author

Chen Liang is a senior software engineer at Uber's interactive analytics team, focusing on Presto. Before joining Uber, Chen was a staff software engineer at LinkedIn's Big Data platform. Chen is also a committer and PMC member of Apache Hadoop. Chen holds two master's degrees from Duke University and Brown University. He is a chapter author of chapter 3.

Book Authors

Dr. Beinan Wang has 15 years of experience in performance optimization and large-scale data processing. He is a PrestoDB committer. Currently a senior staff engineer at Alluxio, he previously led Twitter's Presto team. Dr. Wang earned his Ph.D. in computer engineering from Syracuse University, specializing in distributed systems.

Hope Wang has a decade of experience in Data, AI, and Cloud. An open-source contributor to PrestoDB, Trino, and Alluxio, she also holds AWS Certified Solutions Architect – Professional status. She manages technical marketing at Alluxio and previously worked in venture capital and as a Data Architect. earned a BS in Computer Science, a BA in Economics, and an MEng in Software Engineering from Peking University, as well as an MBA from USC.

About Alluxio

Alluxio, the developer of the open source data platform, makes it easy to manage your data and serve it from any storage to any compute engine in any environment — on premise, in the cloud, or across clouds. By removing complexities and toil from managing and accessing data infrastructure, Alluxio accelerates and future-proofs your data strategy, delivering performant, accessible, cost-effective, resilient, and secure data applications that power improved outcomes, at any scale. To learn more, contact info@alluxio.com or follow us on Linkedin, or Twitter.